State machine

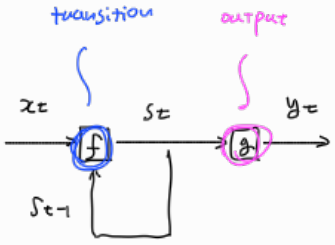
state input output

$(S, X, Y, s_0, f, g)$

$s_0$: init state

$f$: transition function, $S \times X \to S$

$g$: output function, $S \to Y$

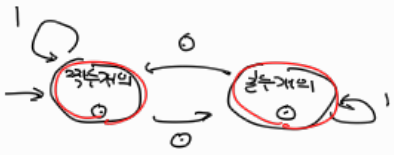



$s_t = f(s_{t-1}, x_t)$

$y_t = g(s_t)$

Note.

- transducer : seq $x$ into seq $y$
- finite state machine




$100111100\ldots\ldots$

---

LTI (linear time-invariant) systems

in discrete time, linear difference equation

$S = \mathbb{R}^m$ (state), $X = \mathbb{R}^l$ (input), $Y = \mathbb{R}^k$ (output)

$y[t] = 3y[t-1] + 6y[t-2] + 5x[t] + 3x[t-2]$

---

Recurrent neural network

- $f(s, x) = f_1(W^{ss} s + W^{sx} x + W_0^{ss})$ — $W^{ss}$: $m \times m$, $W^{sx}$: $m \times l$, $W_0^{ss}$: $m \times 1$
- $g(s) = f_2(W^{o} s + W_0^{o})$ — $W^{o}$: $n \times m$, $W_0^{o}$: $n \times 1$

Note

- non-linear version of LTI system
- learn weight matrices using gradient descent.

---

Markov decision (MDP)

(i) stochastic (확률적) transition

(ii) state equal output

(iii) desirability of state or actions

$= \langle S, A, T, R, \gamma \rangle$ — State, Action; $T$: transition model, $S \times A \times S \to \mathbb{R}$; $R$: reward function, $S \times A \to \mathbb{R}$; $\gamma$: discount $\in [0, 1]$

with policy $\pi : S \to A$.

Note.

- $T(s, a, s') = Pr(S_t = s' \mid S_{t-1} = s, A_{t-1} = a)$
- $R(s, a)$

---

- horizon $H$ : # of steps of interaction

---

horizon $h$ value $V_\pi^h(s)$

(i) $V_\pi^0(s) = 0$

(ii) $V_\pi^1(s) = R(s, \pi(s))$ 기대값.

(iii) $V_\pi^h(s) = \underbrace{R(s, \pi(s))}_{\text{Reward}} + \left( \sum_{s'} \underbrace{T(s, \pi(s), s')}_{\substack{\text{probability} \\ \text{of transition}}} \cdot V_\pi^{h-1}(s') \right)$

Note.

policy $\pi_1$ is "better" at $\pi_2$

$\iff \forall s \in S : V_{\pi_1}^h(s) \geq V_{\pi_2}^h(s)$

$\exists s \in S : V_{\pi_1}^h(s) > V_{\pi_2}^h(s)$

optimal action-value function

Note.

- $Q(s,a)$ = 최적화된 플레이어가 상태 $s$에서 행동 $a$를 선택했을 때 이득
- with DP : $O(mnk)$

(i) $Q^0(s,a) = 0$

(ii) $Q^1(s,a) = R(s,a)$

(iii) $Q^k(s,a) = R(s,a) + \sum_{s'} T(s,a,s') \max_{a'} Q^{k-1}(s',a')$

---

Step 1. Calculate $Q$ : value iteration

Step 2. optimal policy

$$\pi_k^*(s) = \arg\max_a Q^k(s,a)$$

Note.

optimal policy can be multiple.

---

two standard way of dealing with infinite horizon

Note.

- process terminates with prob $= 1-\gamma$
- 미래의 1원보다 현재의 1원이 더 가치가 높음.

(i) average over all time stamps

(ii) discounted infinite horizon

$$\mathbb{E}\left[\sum_{t=0}^{\infty} R_t \mid \pi, S_0\right] = \mathbb{E}\left[R_0 + \gamma R_1 + \gamma^2 R_2 + \cdots \mid \pi, S_0\right]$$

with discount factor $0 < \gamma < 1$

---

$$V_\pi(s) = \mathbb{E}\left[R_0 + \gamma R_1 + \gamma^2 R_2 + \cdots \mid \pi, S_0 = s\right]$$

$$= R(s,\pi(s)) + \gamma \sum_{s'} T(s,\pi(s),s') V_\pi(s')$$

reward / discount / probability of transition

- linear equations that are easy to be solved with Gaussian Elimination
- 높은 값 $= \frac{1}{1-\gamma}$ ~ 1을 넘길 확률

---

MDPs $\Rightarrow$ exists stationary optimal policy $\pi^*$ s.t. for (1) all $s \in S$ (2) all other policies $\pi$

$$\boxed{V_{\pi^*}(s) \geq V_\pi(s)}$$

Note.

value iteration

---

value function : $Q^*(s,a) = R(s,a) + \gamma \sum_{s'} T(s,a,s') \max_{a'} Q(s',a')$

reward / discount / probility of transition

- non-linear and hard to solve
- optimal value function is unique

policy : $\pi^*(s) = \arg\max_a Q^*(s,a)$

- optimal policy is not unique

Value Iteration

Q의 변화폭이 작아질 때까지...

- init $Q_{old}(s,a) = 0$
- repeat (i) $Q_{new}(s,a) = R(s,a) + \gamma \sum_{s'} T(s,a,s') \max_{a'} Q_{old}(s',a')$

(ii) $Q_{old} := Q_{new}$

⇒ terminate

if $\max |Q_{old}(s,a) - Q_{new}(s,a)| < \varepsilon$

---

Properties

- $\pi_Q(s) = \arg\max_a Q(s,a)$
- $\| V_{\pi_{Q_{new}}} - V_{\pi^*} \|_{max} < \varepsilon$
- there is a value $\varepsilon$

such that $|Q_{old} - Q_{new}|_{max} < \varepsilon \Rightarrow \pi_{Q_{new}} = \pi^*$

- $\| V_{\pi_{Q_{new}}} - V_{\pi^*} \|_{max}$ 는 단조감소.
- can be executed asynchronously

비동기식

i.e enough for every $(s,a)$'s

to be infinitely updated.